REMA — Interview Demo Guide

Rapid Emergency Medical Access | Medical Logistics in a Sinking City

University Track | Vietnam Red Cross Challenge | Solo Build

Node.js React PostgreSQL Socket.io JWT + Refresh Claude AI

Frontend	`https://rema-system.vercel.app`
Backend API	`https://rema-medical-logistics.onrender.com`
Swagger Docs	`https://rema-medical-logistics.onrender.com/api/docs`

Test accounts (all password: rema1234)

Email	Role	Password
admin@rema.kh	SUPER_ADMIN	rema1234
coordinator@rema.kh	EMERGENCY_COORDINATOR	rema1234
hub1@rema.kh	HUB_MANAGER (Dangkao)	rema1234
volunteer1@rema.kh	VOLUNTEER	rema1234
viewer@rema.kh	VIEWER (read-only)	rema1234

Pre-demo reset Before every demo: log in as admin@rema.kh and hit POST /api/alert/reset via Swagger (or the Reset button on Dashboard) to put the system back in Phase 0 standby. This preserves all DB records — it just resets the activation state.

01 Project Overview

What to say in the first 60 seconds

The one-sentence pitch

REMA is a full-stack emergency medical logistics system that gets the right supplies to the right people — even when a city is flooded — by pre-positioning stock before roads close, scoring households by medical vulnerability, and adapting delivery mode to water depth in real time.

Solo build context (say this early)

Key differentiator This is a 4-person team challenge that I built alone. Backend, frontend, database design, DevOps, CI/CD, AI integration, strategy documents — everything. 22 structured chat sessions over the course of the project.

System scope (rattle these off confidently)

What	How / Why it matters
API endpoints	55+ REST endpoints across 12 resource groups
Database tables	17 tables — full relational schema with audit trails
Frontend views	10 views — role-gated, mobile-optimized for volunteers
User roles	5 roles with full RBAC (SUPER_ADMIN down to VIEWER)
Unit tests	113 tests — pure utility functions, no DB or HTTP
Documented assumptions	64 — judges reward honest assumptions over false precision
Annual system cost	~$69,500 USD — real budget with EMK supply cost breakdown
Cost per beneficiary	~$0.95/person/year — competitive with SE Asia humanitarian programs

02 Architecture Deep Dive

The technical decisions worth explaining

Three-layer logistics model

The core insight driving every technical decision: reactive logistics fails in urban flooding because by the time demand is confirmed, roads are already gone. REMA pre-positions supplies before flooding peaks.

Layer	What	Tech involved
Layer 1 - Central Warehouse	Master stock, 30% reserve after dispatch, sourcing coordination	POST /api/stock/dispatch, stock_movements audit table
Layer 2 - Sub-Warehouses x3	One per district, stocked Hours 3-8 before flooding peaks	districts + sub_warehouses + stock tables, scarcity logic
Layer 3 - Last Mile	Motorbike / bicycle / boat tiered by water depth to 80cm	routes table, getDeliveryModeForDepth() utility, per-zone recommend API

Backend architecture decisions

What	How / Why it matters

Node.js + TypeScript + Express	Type-safe throughout. Prisma ORM gives type-safe DB queries with migration history. No raw SQL.
PostgreSQL on Supabase	Managed DB in Singapore region. Connection pooling handled. Free tier with 500MB — enough for demo scale.
JWT access token (15 min)	Short-lived to limit stolen token exposure. Forces refresh cycle — no 'login once, valid forever'.
httpOnly refresh token (7 days)	Stored in httpOnly cookie — JS can't read it, so XSS attacks can't steal it. Hash stored in DB, raw token never persisted.
SHA-256 token hashing	If DB is compromised, attacker gets hashes not tokens. Same pattern as password hashing with bcrypt.
Socket.io for real-time	Three events: phase_changed, scarcity_triggered, incident_reported. Broadcast to all connected clients. Dashboard silently refetches on any event.
In-memory cache	Dashboard summary cached for 15s, district summaries for 10s. Invalidated on phase change or stock dispatch. Avoids hammering DB on every poll.
30s polling fallback	WebSocket is primary. If socket drops, polling catches updates. Redundancy over elegance — core operating principle.

Frontend architecture decisions

What	How / Why it matters
React + Vite + TypeScript	Vite for fast HMR in dev. TypeScript shared types with backend where possible (scoring utility is identical in both).
Tailwind CSS	Utility-first — no custom CSS files. Consistent spacing and color tokens.
Role-based routing	JWT role decoded client-side. ProtectedRoute wrapper checks role before render. 5 roles, each sees a different nav sidebar.
Scoring engine duplicated	backend/src/utils/scoring.ts and frontend/src/utils/scoring.ts are identical. Live preview in volunteer form doesn't need an API call. Server validates on submit.

Axios interceptor with queue	On 401: pause all in-flight requests, call /api/auth/refresh, replay queued requests. User never sees a login prompt mid-session.
mustChangePassword redirect	Admin-created users land on /change-password. Navigation blocked until flag is cleared. Enforced client-side as UX gate.
Avatar: client-side resize	Canvas API resizes to 128x128 JPEG before upload. Max ~25kb. Stored as base64 in users table — no external file storage dependency.
Leaflet.js for routing map	Real OSM district boundaries. Sutherland-Hodgman algorithm clips polygons into per-zone areas. Per-zone depth sliders connected to POST /api/route/update.

03 Demo Walkthrough

9 steps, ~15 minutes, in this exact order

Setup tip Open two browser tabs before you start: Tab 1 = frontend (rema-system.vercel.app), Tab 2 = Swagger docs. This lets you show the API contract alongside the UI without context switching.

1 Reset + Login as Admin

Login as: **admin@rema.kh**

• Go to Swagger, POST /api/alert/reset — confirm 200 OK. This puts the system in Phase 0 standby without clearing any DB records.

• Log in to the frontend. Show the V9 User Management page — the only page accessible to SUPER_ADMIN that others can't see.

• Create a new HUB_MANAGER user. Show the mustChangePassword flow: new user is forced to change password on first login.

• Show the Sidebar user card at the bottom — click it to go to the Profile page. Upload an avatar, explain client-side resize.

Technical talking points:

→ SUPER_ADMIN accounts are created via seed script only — never via API. Deliberate security decision: no API endpoint means no way to escalate privileges via a compromised token.

→ mustChangePassword is in the JWT payload and login response. Frontend reads it and blocks navigation to /profile or /dashboard until cleared.

→ avatarBase64 is stored directly in the users table. No S3, no CDN — keeps the system self-contained with zero external service dependencies for user data.

2 Trigger Flood Activation

Login as: **coordinator@rema.kh**

• Log in as coordinator. Show the Phase 0 banner — system is in standby.

• Show the trigger conditions panel: warningLevelTwo, rainfallExceeds100mm, streetFloodingReport. Explain any 2 of 3 must be true simultaneously.

• Tick warningLevelTwo + rainfallExceeds100mm, hit Trigger. System activates to Phase 1.

• Point out the phase banner changes color. If you have a second browser tab open (or another device), show the WebSocket event arriving in real time.

• Show notifications: HUB_MANAGERs and ECs receive phase_changed notification automatically.

Technical talking points:

→ 2-of-3 activation logic is a pure utility function (shouldActivate()) with 13 unit tests covering all 8 boolean combinations. No single person can trigger the system alone.

→ socket.io emits phase_changed to all connected clients. DashboardPage subscribes and refetches silently — no page reload needed.

→ Notification records are written to the notifications table on phase advance. The GET /api/notifications endpoint returns unread count + messages per user.

→ Cache invalidation: getDashboardSummary cache is cleared on phase advance so next poll reflects new state immediately.

3 Dashboard - Operations View

Login as: **coordinator@rema.kh**

• Show the V1 Operations Dashboard. Walk through each panel: Phase banner, District summary cards, Stock levels chart (Recharts), Priority queue table, Active incidents panel.

• Click the AI Brief button. Explain: server reads aggregate dashboard state, sends to Claude API, returns a 3-part brief: situation summary, priority alert, recommended next step.

• Point out the 'Advisory only - human decision required' banner. The AI endpoint is read-only — it cannot trigger any DB write.

• Show the 30-second auto-refresh indicator and the manual refresh button.

Technical talking points:

→ AI Brief uses @anthropic-ai/sdk server-side only. ANTHROPIC_API_KEY is never in the frontend bundle or any repo file. Graceful 503 degradation if API is unavailable.

→ No PII in the AI prompt — only aggregate counts (totalCritical: 12, scarcityActive: false, phase: 1). Even if the prompt were leaked, no household data is exposed.

→ Dashboard summary has 15s TTL in-memory cache. isInScarcity() is a pure utility (18 unit tests). Scarcity triggers at exactly 30% of original allocation — not current max, the original.

→ Recharts renders EMK-1/2/3 remaining per district as a grouped bar chart. Data comes from GET /api/dashboard/summary which aggregates across all sub_warehouses in the district.

4

Routing Map

Login as: **coordinator@rema.kh**

• Navigate to V2 Routing Map. Show the Leaflet.js map with the 3 district polygons (Dangkao, Mean Chey, Pou Senchey).

• Each district is split into 3 zones (A/B/C) — 9 zone polygons total. Explain the Sutherland-Hodgman lat-band clipping algorithm used to split OSM boundaries.

• Drag a zone depth slider above 80cm — the zone turns red and shows SUSPENDED with a pulsing border.

• Drag a slider to 45cm — show it switches from MOTORBIKE to BICYCLE_OR_FOOT mode automatically.

• Show the route change log below the map — every depth update is recorded with who changed it and when.

Technical talking points:

→ Zone polygons use real OSM district boundary data, not rectangles. Lat-band splitting gives realistic polygon shapes that follow actual district edges.

→ GET /api/route/recommend?districtId=X returns per-zone array: [{ zone: 'A', waterDepthCm: 45, deliveryMode: 'BICYCLE_OR_FOOT', active: true }]. Old district-level averaging was removed as operationally misleading.

→ getDeliveryModeForDepth() is a pure utility with 23 unit tests — all 4 tiers and all 3 boundary values tested. 0-30: MOTORBIKE, 30-60: BICYCLE_OR_FOOT, 60-80: BOAT, >80: SUSPENDED.

→ route_logs table records every depth change with previous/new depth, previous/new mode, and the user who made the change. Full audit trail.

→ SUSPENDED zones emit scarcity_triggered socket event. All dashboard clients see the warning without refreshing.

5

Hub Manager Portal

Login as: **hub1@rema.kh**

• Log in as hub1@rema.kh. They can only see their own district (Dangkao) — no cross-district data.

• Show V7 Hub Manager Portal. Walk the 5 tabs: Stock / Volunteers / Deliveries / Incidents / Radio.

• Stock tab: dispatch some EMK-1 kits. Show a stock movement record being created in the movements log. Explain the movements table is the audit trail.

• Volunteers tab: add a volunteer, assign them to a zone and team number.

• Deliveries tab: start a delivery run. Show the run status (IN_PROGRESS).

• Radio tab: submit a check-in for the 08:00 slot. Show the compliance panel.

Technical talking points:

→ HUB_MANAGER can only read/write their own districtId. The RBAC middleware checks JWT role and districtId on every write endpoint. EC and SUPER_ADMIN can act across all districts.

→ Every stock change (dispatch, reallocate, adjust) writes a stock_movements record with emkType, quantity, movementType, reason, and performedBy (userId). The DB never loses stock history.

→ Scarcity check runs inside POST /api/stock/dispatch: if any EMK type falls below 30% of its original total after the dispatch, it emits scarcity_triggered via socket.io and creates notifications for EC + SUPER_ADMIN.

→ radio_checkins has 4 fixed scheduledTime slots per day (T0800, T1200, T1600, T2000). GET /api/radio/compliance returns which districts have checked in for each slot today — EC can see compliance at a glance.

→ Volunteer records are linked to user accounts (VOLUNTEER role). Community volunteers with no system login can also be created via POST /api/volunteers — two-path design.

6

Volunteer Mobile View

Login as: **volunteer1@rema.kh**

• Switch to a phone or narrow the browser. Log in as volunteer1@rema.kh.

• Show V8 Volunteer Mobile View — bottom nav bar with large touch targets: Assessment / Delivery / Incident.

• Assessment tab: fill in a household assessment form. Show the live score updating as you answer each category question.

• Submit — show the household appearing in the priority queue on the dashboard (you may need to switch tabs).

• Delivery tab: find a CRITICAL household, submit a delivery receipt. Show EMK-3 and EMK-2 as separate line items for a household with both chronic illness and a vulnerable member.

• Incident tab: quickly report a ROUTE_BLOCKED incident. Switch to the coordinator tab — show it appearing in the incidents panel.

Technical talking points:

→ The 20-point scoring engine (scoreHousehold() + assignBand() + recommendEmk()) runs identically in frontend TypeScript (live preview) and backend TypeScript (validation on submit). 59 unit tests cover all scoring rules.

→ Category 1 (medical urgency) has max 8 points — heavier than all other categories combined. This is the key design choice: chronic illness medication loss is immediately life-threatening.

→ A household with both chronic illness AND a vulnerable member gets EMK-3 AND EMK-2. The bug we fixed: an earlier version gave only EMK-3 by mistake. Now emk3 and emk2 are independent flags in calculateEmkQuantity().

→ POST /api/delivery/receipts accepts kits[] array for multi-type delivery. Each kit type deducts from its own stock bucket in a single transaction. stock_movements records one entry per kit type.

→ incidents table supports 5 types: ROUTE_BLOCKED, VOLUNTEER_SAFETY, STOCK_SCARCITY, BUILDING_FLOODED, OTHER. VOLUNTEER_SAFETY and BUILDING_FLOODED incidents also notify EC + SUPER_ADMIN via socket.io.

7

Stock Reallocation + Scarcity Mode

Login as: **coordinator@rema.kh**

• Log in as coordinator. Go to Dashboard and dispatch a large batch of EMK-1 from one district's sub-warehouse until it hits below 30%.

• Watch the scarcity_triggered WebSocket event fire — the stock chart in the dashboard updates in real time.

• Show the notification bell — EC and SUPER_ADMIN received a scarcity notification.

• Now use POST /api/stock/reallocate (EC-only endpoint) to move stock from another district to cover the shortage.

• Show the stock movements log — every action is recorded with movementType: REALLOCATION.

Technical talking points:

→ Scarcity is checked after every stock_dispatch call. isInScarcity() compares emkNRemaining against emkNTotal (the current activation baseline, not a fixed historical figure). This prevents false positives after a resupply tops up the total.

→ Cross-district reallocation is EMERGENCY_COORDINATOR-only — HUB_MANAGER can only adjust within their own sub-warehouse. The role boundary is enforced in middleware.

→ scarcity_triggered socket event includes { districtId, emkType, percentRemaining } in the payload. Dashboard components can react specifically to the affected district.

→ POST /api/stock/reallocate creates two stock_movements records: one REALLOCATION (negative quantity) for the source, one REALLOCATION (positive) for the destination. Net zero — no EMKs created or destroyed.

8

Phase Advance + System Reset

Login as: **coordinator@rema.kh**

• Show the Phase Controls panel on the dashboard (EC and SUPER_ADMIN only). Advance from Phase 1 to Phase 2.

• Explain Phase 2: adaptive last-mile delivery from sub-warehouses. Phase direction is forward-only in the UI — prevents accidental rollback during active response.

• Switch to admin tab. Show POST /api/alert/reset in Swagger — explain this is SUPER_ADMIN only and resets the flood_alerts state without deleting any DB records.

• After reset: phase banner returns to Phase 0. All historical households, deliveries, stock movements, and incidents are preserved for post-flood audit.

Technical talking points:

→ PATCH /api/alert/phase accepts phase: 1 or 2 only — no backwards movement. The backend rejects any phase value lower than or equal to the current phase.

→ POST /api/alert/reset sets activated: false and phase: 0 on the active flood_alerts record. It does NOT delete the record — assumption 57: only state is reset, not history.

→ Phase advance emits phase_changed via socket.io and creates notifications for all HUB_MANAGERs, EC, and SUPER_ADMIN in a single DB transaction.

→ After reset: GET /api/alert/status returns { activated: false, phase: 0 }. All other data intact. Good for running multiple demo flows back-to-back.

9

Testing + CI/CD Pipeline

Login as: **N/A (show terminal or GitHub Actions)**

• Open the GitHub repository. Show the .github/workflows/ci.yml file.

• Explain: every PR runs the full test suite before merge. Every push to main runs tests, then triggers Render deploy hook.

• Show the 4 test files and their scope: scoring (59 tests), stock utils (18), alert (13), route (23) = 113 total.

• Emphasize: all tests are pure utility functions — no Prisma, no HTTP, no database. Fast, deterministic, no test DB needed.

• Show one test file (scoring.test.ts) — point to the Section C worked example reproduced as a test case.

Technical talking points:

→ 113 tests guard all business-critical logic: the 20-point scoring system, the 30% scarcity threshold, the 2-of-3 activation trigger, and the 4-tier water depth routing.

→ CI uses Node.js 20 with npm ci (clean install from lock file, not npm install). Reproducible builds.

→ Render deploy hook is stored as a GitHub Actions secret (RENDER_DEPLOY_HOOK_URL). The workflow calls it only after tests pass — no test failure can deploy a broken build.

→ Vercel autodeploys the frontend from GitHub main — no manual step. Backend and frontend deploy independently.

→ ts-jest compiles TypeScript test files without needing a separate tsc step. Jest configuration in jest.config.ts.

04 Technical Talking Points

What to say when they ask 'tell me about X'

If they ask about authentication / security

What	How / Why it matters
Token strategy	15-minute access token + 7-day httpOnly refresh token. Short window limits stolen token exposure. Refresh is transparent — axios interceptor catches 401, calls /api/auth/refresh, replays the original request.
Refresh token storage	Raw token goes to the client in a httpOnly cookie (JS can't read it). A SHA-256 hash of the token is stored in the refresh_tokens table. DB compromise gives you hashes, not usable tokens.

Parallel request handling	When the access token expires mid-session, multiple requests might 401 simultaneously. The axios interceptor queues them all, refreshes once, then replays the queue. No duplicate refresh calls.
Password security	bcrypt hashing for passwords. Admin-created users get mustChangePassword: true — they're forced to set their own password before accessing any feature.
RBAC	5 roles enforced in Express middleware on every route. HUB_MANAGER writes are checked against their districtId claim in the JWT — can't modify other districts even with a valid token.
SUPER_ADMIN creation	No API endpoint to create SUPER_ADMIN. Seed script only. If there's no endpoint, there's no way to escalate via a compromised token.

If they ask about real-time / WebSocket

What	How / Why it matters
Socket.io setup	Express app is wrapped in http.Server. socket.io Server is initialized on that HTTP server. Both REST and WebSocket share the same port.
Three events	phase_changed (on activation or phase advance), scarcity_triggered (when any district stock drops below 30%), incident_reported (on new incident creation). That's it — minimal surface area.
Broadcast vs. targeted	Events are broadcast to all connected clients — no per-user or per-district filtering. All operational roles need system-wide situational awareness. Filtering adds complexity for negligible security benefit in this context.
Dashboard integration	DashboardPage subscribes to all 3 events in a useEffect. On any event, it calls the refetch function. Silent background update — user sees new data without knowing why.
Fallback	30-second polling remains as a fallback. If WebSocket drops (Render free tier sleeps, network hiccup), the dashboard still catches up within 30 seconds.

If they ask about caching

What	How / Why it matters

Where it lives	In-memory cache utility in backend/src/utils/cache.ts. A simple Map with TTL. Not Redis — intentionally simple for this scale.
What is cached	getDashboardSummary (15-second TTL) and getDistrictDashboard (10-second TTL). These are the most frequently polled endpoints.
Invalidation strategy	Targeted invalidation — not cache-bust-everything. Phase advance clears dashboard summary. Stock dispatch clears both summary and the affected district. Route update doesn't clear dashboard (it has its own endpoint).
Why not Redis	Render free tier, self-contained deployment, demo scale. Redis adds a service dependency and monthly cost. The in-memory cache is sufficient for the polling frequency. Would swap to Redis for production scale.
Cache key design	'dashboard:summary' and 'dashboard:district:{id}'. Simple string keys. Easy to invalidate a specific district without clearing others.

If they ask about the scoring system

What	How / Why it matters
20-point scale	5 categories: medical urgency (max 8), household vulnerability (max 5), flood exposure (max 4), self-sufficiency (max 2), isolation (max 1). Medical urgency is weighted heavier than all others combined.
Score bands	15-20 = CRITICAL (deliver in current run), 10-14 = HIGH (same day), 5-9 = MEDIUM (within 48h), 0-4 = STANDARD (community collection point). Not a percentage — fixed thresholds.
EMK recommendation	Separate from scoring. EMK-3 if chronicIllness AND medicationLost. EMK-2 if vulnerable household flag. A household can get both — they address independent medical needs. Bug we fixed: an earlier version skipped EMK-2 when EMK-3 was assigned.
Tiebreakers	1. Higher Category 1 score. 2. Infant under 6 months. 3. First assessment form submitted. Fully mechanical — no volunteer discretion.

Fairness safeguards	Cross-ward volunteer assignment (reduces favoritism). Written scores on paper forms (auditable). Score challenge mechanism (household can contest). Collection points always open regardless of score (no permanent exclusion).
Unit tests	59 tests covering: all scoring category combinations, score band edge cases (14 vs 15 boundary, 9 vs 10), EMK recommendation logic, the Section C worked example reproduced exactly, and input validation.

If they ask about the database design

What	How / Why it matters
17 tables	users, districts, sub_warehouses, stock, stock_movements, flood_alerts, households, household_assessments, volunteers, volunteer_assignments, delivery_runs, delivery_receipts, routes, route_logs, incidents, radio_checkins, notifications, refresh_tokens.
Audit trail design	stock_movements records every stock change with movementType, quantity, reason, performedBy, and createdAt. route_logs records every depth change. Never delete records — deactivate users, log movements, resolve incidents.
Stock semantics	emkNTotal is the current activation's dispatch baseline. emkNRemaining tracks what's left. Scarcity is emkNRemaining / emkNTotal < 0.30. When stock is resupplied, Total is updated — preventing false scarcity positives.
Soft deletes	Users are never deleted — set active: false. Historical delivery receipts still reference the userId. Departing staff don't break audit trails.
Prisma migrations	Every schema change has a named migration file. Migration history is in version control. Supabase applies them on deploy.

05 What Makes This Stand Out

The things worth highlighting explicitly

Domain-driven design	Every technical decision traces back to a real operational constraint. EMK-3 is never pre-stored at sub-warehouses because community buildings can't maintain 2-8C cold chain. Scarcity triggers at exactly 30% because Section C defines it. The 2-of-3 activation trigger prevents any single person from triggering the system. The DB schema reflects the logistics model — not the other way around.
Comprehensive audit trail	Every stock change, route update, delivery, and incident is logged with who, what, when, and why. Nothing is ever deleted. This is a humanitarian system — accountability is non-negotiable. stock_movements, route_logs, delivery_receipts, and incidents are all append-only audit tables.
Security done properly	httpOnly refresh tokens with SHA-256 hashing. 15-minute access token window. Parallel 401 request queuing. mustChangePassword enforcement. No API endpoint for SUPER_ADMIN creation. RBAC checked in middleware, not in route handlers. These aren't afterthoughts — they're in the initial design.
Real-time + fallback	WebSocket for instant updates. 30-second polling as a fallback. In-memory cache to avoid hammering the DB. The system works if WebSocket drops, if the cache is cold, if the network is slow. Redundancy over elegance — it's in the operating principles.
64 documented assumptions	Every design decision where a real-world constraint was assumed is documented. Judges at the Vietnam Red Cross challenge reward this — it shows you understand the difference between what you built and what you know. EMK-3 cold chain, boat owner agreements, MoH medication release timing, volunteer availability buffers — all logged.
Solo built in 22 sessions	No team. 55+ API endpoints, 17 DB tables, 10 frontend views, 113 unit tests, CI/CD pipeline, AI integration, draw.io diagrams, a print-ready operating protocol PDF, and a complete 6-section strategy document. Each session was structured with an opening ritual (read HANDOFF.md + PROJECT_SCOPE.md) and a closing ritual (update all 4 docs + git commit). Disciplined project management, not improvisation.

06 Common Interview Questions

Answers you can give with confidence

Q: Why not use Redis for caching?

For this scale and deployment target (Render free tier), in-memory is sufficient. Redis adds a managed service cost and a network hop. The TTL-based Map gives the same hit rate for 30-second polling. I'd swap to Redis if moving to production with horizontal scaling — at that point, multiple instances would have separate in-memory caches and that's a problem.

Q: Why store avatar as base64 in the DB instead of S3?

Self-contained deployment with zero external service dependencies for user data. 128x128 resized JPEG is ~15-25kb per user — negligible for a 50-user system. The Supabase free tier has 500MB. If this were a 50,000-user product, yes, S3 or Cloudflare R2. For this use case it's the right trade-off.

Q: Why is the scoring engine duplicated in both frontend and backend?

Live preview in the volunteer assessment form requires the score to update as you fill in each field. An API call per field would add 200-500ms latency and hammers the backend. The duplicate is a deliberate choice — frontend uses it for UX, backend validates on submit. The 59 unit tests run against the backend version, which is the source of truth.

Q: Why socket.io broadcast instead of per-user or per-district rooms?

All operational roles (EC, Hub Manager, Volunteer) need system-wide situational awareness. A Hub Manager needs to know if another district's stock goes critical — they might be asked to reallocate. Filtering adds complexity and the risk of someone missing a critical alert. The event payload includes districtId so clients can choose how to display it.

Q: How would you scale this beyond Render free tier?

Backend to Render paid tier or Railway (once budget allows). Redis for distributed caching across multiple instances. PostgreSQL to a dedicated Supabase paid plan with connection pooling. WebSocket would need sticky sessions (already supported by socket.io with Redis adapter). The architecture supports it — nothing is fundamentally single-instance-only.

Q: What would you add if you had more time?

Server-side mustChangePassword enforcement (currently a UX gate, not a security gate). Per-district WebSocket rooms. Offline-first PWA for the volunteer mobile view (it currently requires connectivity). EMK-3 cold chain transfer tracking as a separate sub-flow. End-to-end tests with Playwright against the live Vercel deployment.

Q: How did you manage a project this large alone?

22 structured sessions, each with a fixed opening ritual (read HANDOFF.md + PROJECT_SCOPE.md) and closing ritual (update 4 docs + git commit). Every session had a single clearly defined scope. No session ended without updating the handoff doc — that meant the next session always started with a complete, accurate picture of the system state.

Q: What's the hardest bug you fixed?

The EMK quantity bug: households with both chronic illness and a vulnerable member should receive EMK-3 AND EMK-2 independently. An earlier condition (emk3 === 0 guard on EMK-2 assignment) caused EMK-2 to be skipped whenever EMK-3 was present. The bug was in both backend scoring.ts and frontend scoring.ts — two files that must stay identical. Finding it required tracing from delivery receipt submission back through the scoring engine to the calculateEmkQuantity function.

07 Quick Reference

The numbers and facts to have ready

API endpoints: 55+
DB tables: 17
Frontend views: 10
User roles: 5
Unit tests: 113
Test files: 4 (scoring, stock, alert, route)
CI/CD: GitHub Actions — PR blocks on test fail
Access token TTL: 15 minutes
Refresh token TTL: 7 days (httpOnly cookie)

Backend hosting: Render (Node.js)
DB hosting: Supabase PostgreSQL, Singapore
Frontend hosting: Vercel
Real-time: socket.io, 3 event types
Cache TTL (dashboard): 15 seconds
Cache TTL (district): 10 seconds
Scarcity threshold: 30% of original allocation
Activation trigger: Any 2 of 3 conditions
Water depth cutoffs: 30 / 60 / 80 cm

EMK types at a glance

Type	Target	Key rule	Storage
EMK-1	General household	ORS, wound care, paracetamol, hygiene	Central + sub-warehouses
EMK-2	Vulnerable (elderly, pregnant, infant, disabled)	All EMK-1 + infant formula, thermometer, BP card	Central + sub-warehouses
EMK-3	Chronic illness (lost medication access)	3-day meds, glucose strips, syringes, referral card	MoH cold storage ONLY — never sub-warehouses

Delivery mode tiers

Water Depth	Delivery Mode	Carry Capacity
0-30 cm	Motorbike with waterproof pannier bags	~25kg, 5x EMK-1 or 3x EMK-2
30-60 cm	Cargo bicycle or volunteer on foot	~10-20kg, 2-4x EMK-1
60-80 cm	Small motorized boat or inflatable raft	~150kg, 30x EMK-1 or 20x EMK-2
>80 cm	SUSPENDED - escalate to civil defense	No delivery - volunteer safety hard limit

The 3-point summary The 3 things interviewers remember: (1) solo built a 4-person challenge, (2) every technical decision traces back to a real humanitarian constraint, (3) 113 unit tests and full CI/CD guard business-critical logic.